Function

In [3]:
```
def greet_user():
    """display a simple greeting."""
    print("Hello world!")
```

In [4]:
```
greet_user()
```

```
Hello world!
```

In [16]:
```
def greet_user(username):
    """display a simple greeting."""
    print("Hello, "+username.title() +"! ")
```

In [17]:
```
greet_user('mica')
```

```
Hello, Mica!
```

Excercise

In [33]:
```
# 8-1
def display_message():
    print("I have learned function!")
```

In [34]:
```
display_message()
```

```
I have learned function!
```

In [35]:
```
# 8-2
def favorite_book(title):
    print("One of my favorite books is "+title.title()+".")
```

In [36]:
```
favorite_book('My heart will go on')
```

```
One of my favorite books is My Heart Will Go On.
```

Passing arguments

1. Positional Arguments

In [46]:
```
def describe_pet(animal_type,pet_name):
    print("\nI have a "+ animal_type + ".")
    print("My " + animal_type + "'s name is " + pet_name.title() + ".")
```

In [48]:
```
describe_pet('cat','miky')
describe_pet('dog','lily')
describe_pet('lily','dog') # check order
```

```
I have a cat.
My cat's name is Miky.

I have a dog.
My dog's name is Lily.

I have a lily.
My lily's name is Dog.
```

2. Keyword Arguments

In [31]:
```python
def describe_pet(animal_type,pet_name):
    print("\nI have a "+ animal_type + ".")
    print("My " + animal_type + "'s name is " + pet_name.title() + ".")

describe_pet(animal_type='hamster', pet_name='harry')
describe_pet('monkey', pet_name='king')
describe_pet(pet_name='lily',animal_type='dog')
describe_pet('bird', 'sunny')

# 下面这两个会报错，因为不能指定最后一个参数
# describe_pet(pet_name='lily','dog') 违反关键词规则，关键词必须放在后面
# describe_pet('lucy',animal_type='bird') 指定了type，但是按顺序lucy有给type了，
```

```
I have a hamster.
My hamster's name is Harry.

I have a monkey.
My monkey's name is King.

I have a dog.
My dog's name is Lily.

I have a bird.
My bird's name is Sunny.
```

3. Default Values

In [61]:
```python
# 赋值的参数一定要写到最后面，如果有赋值会按赋值输出，没有按默认值输出
def describe_pet(pet_name,animal_type='dog'):
    print("\nI have a "+ animal_type + ".")
    print("My " + animal_type + "'s name is " + pet_name.title() + ".")

describe_pet('rainy')
describe_pet('rainy','cat')
```

```
I have a dog.
My dog's name is Rainy.

I have a cat.
My cat's name is Rainy.
```

Excercise

In [33]:
```
# 8-3
def make_shirt(size,message):
    print("Size_"+size+" ," + message + ".")

#  positional arguments
make_shirt('L','This shirt is for man')

# keyword arguments
make_shirt(message='This shirt is for woman',size='M')
```

```
Size_L ,This shirt is for man.
Size_M ,This shirt is for woman.
```

In [79]:
```
# 8-4
def make_shirt(size='Large',message='I love Python'):
    print("Size_"+size+" ," + message + ".")

# Make a large shirt and a medium shirt with the default message
make_shirt()
make_shirt('Medium')

# other size
make_shirt('Small','This shirt is for man')
```

```
Size_Large ,I love Python.
Size_Medium ,I love Python.
Size_Small ,This shirt is for man.
```

In [93]:
```
# 8-5
def describe_city(city,country='China'):
    print(city.title() + " is in " + country.title() + ". This is a beautiful ci

# 1st city
describe_city('beijing')

# 2nd city
describe_city(city='bangi',country='malaysia')

# 3rd city
describe_city(country='america',city='newyork')

# 4th city
describe_city('london',country='england')
```

```
Beijing is in China. This is a beautiful city.
Bangi is in Malaysia. This is a beautiful city.
Newyork is in America. This is a beautiful city.
London is in England. This is a beautiful city.
```

Return Values

In [97]:
```
def get_formatted_name(first_name,last_name):
    full_name=first_name + " " +last_name
    return full_name.title()

musician=get_formatted_name('mica','liang')
print(musician)
```

```
Mica Liang
```

In [98]:
```
def get_formatted_name(first_name,middle_name,last_name):
    full_name=first_name + " " + middle_name + " " +last_name
    return full_name.title()

musician = get_formatted_name('john', 'lee', 'hooker')
print(musician)
```

```
John Lee Hooker
```

Making an Argument Optional

In [101…
```
# 注意赋值的一定要放在最后
def get_formatted_name(first_name,last_name,middle_name=''):
    if middle_name:
        full_name=first_name + " " + middle_name + " " +last_name
    else:
        full_name=first_name + " " +last_name
    return full_name.title()

musician=get_formatted_name("mica","liang")
print(musician)

musician = get_formatted_name('john', 'hooker', 'lee')
print(musician)
```

```
Mica Liang
John Lee Hooker
```

def + dictionary

In [103…
```
def build_person(first_name,last_name):
    person={'first':first_name,'last':last_name}
    return person

musician=build_person('mica','liang')
print(musician)
```

```
{'first': 'mica', 'last': 'liang'}
```

In [110…
```
# 薄弱题
def build_person(first_name,last_name,age=''):
    person={'first':first_name,'last':last_name}
    if age:
        person['age']=age
    return person

musician=build_person('mica','liang',age=27)
print(musician)
```

```
{'first': 'mica', 'last': 'liang', 'age': 27}
```

In [7]:
```
# 同类练习题1
def build_student(name,class_num,age=''):
    student={'student_name':name,'student_class':class_num}
    if age:
        student['age']=age
    return student
```

```
student1=build_student('jessica','H101',18)
print(student1)

student2=build_student('mica','H103')
print(student2)
```

```
{'student_name': 'jessica', 'student_class': 'H101', 'age': 18}
{'student_name': 'mica', 'student_class': 'H103'}
```

In [4]:

```
# 同类练习题2
def build_book(name,author,price=''):
    book={'book_name':name,'author_name':author}
    if price:
        book['price']=price
    return book

book1=build_book('Animal Farm','George Orwell',30)
print(book1)
```

```
{'book_name': 'Animal Farm', 'author_name': 'George Orwell', 'price': 30}
```

def + while

In [117…

```
def get_formatted_name(first_name,last_name):
    full_name=first_name + " " +last_name
    return full_name.title()

while True:
    print("\nPlease tell me your name:")
    print("enter 'q' at any time to quit.")

    f_name=input("First name:")
    if f_name == 'q':
        break

    l_name=input("Last name:")
    if l_name == 'q':
        break
    formatted_name=get_formatted_name(f_name,l_name)
    print("\nHello, " + formatted_name + "!")
```

```
Please tell me your name:
enter 'q' at any time to quit.
Hello, Mica Liang!

Please tell me your name:
enter 'q' at any time to quit.
```

Excercise

In [128…

```
# 8-6 返回值
def city_country(city,country):
    return city.title()+","+country.title()

print(city_country('shanghai','china'))
```

```
print(city_country('kuala lumpur','malaysia'))
print(city_country('newyork','america'))
```

```
Shanghai,China
Kuala Lumpur,Malaysia
Newyork,America
```

In [137…

```
# 8-7 def+ default value 薄弱题
def make_album(name,title, track_number=''):
    if track_number:
        music_album={'album_name':name,'album_title':title,'track':track_number}
    else:
        music_album={'album_name':name,'album_title':title}
    return music_album

musician1=make_album('jessica','Love story',30)
print(musician1)

musician2=make_album('angle','My heart will go on')
print(musician2)

musician3=make_album('wenda','Leave me alone',20)
print(musician3)
```

```
{'album_name': 'jessica', 'album_title': 'Love story', 'track': 30}
{'album_name': 'angle', 'album_title': 'My heart will go on'}
{'album_name': 'wenda', 'album_title': 'Leave me alone', 'track': 20}
```

In [9]:

```
# 8-7 同类练习题
def make_movie(director,name,time=''):
    if time:
        movie={'director_name':director,'movie_name':name,'time':time}
    else:
        movie={'director_name':director,'movie_name':name}
    return movie

movie1=make_movie('Robert Zemeckis','Forrest Gump','2.2 hours')
print(movie1)

movie2=make_movie('James Cameron','Titanic')
print(movie2)
```

```
{'director_name': 'Robert Zemeckis', 'movie_name': 'Forrest Gump', 'time': '2.2 h
ours'}
{'director_name': 'James Cameron', 'movie_name': 'Titanic'}
```

In [5]:

```
# 8-8 def + while + 返回字典 + quit退出循环  薄弱题
def make_album(name,title):
    music_album={'album_name':name,'album_title':title}
    return music_album

while True:
    print("enter 'q' at any time to quit.")

    artist_name=input("Album's name:")
    if artist_name=='q':
        break

    song_title=input("Song's title:")
    if song_title=='q':
        break
```

```
    album=make_album(artist_name,song_title)
    print(album)
```

```
enter 'q' at any time to quit.
{'album_name': 'mica', 'album_title': 'love story'}
enter 'q' at any time to quit.
```

In [10]:

```
# 8-8 同类练习题
def make_pet(type,name):
    pet={'pet_type':type,'pet_name':name}
    return pet

while True:
    print("enter 'q' at any time to quit.")
    animal_type=input("tell me you animal's type:")
    if animal_type=='q':
        break

    print("enter 'q' at any time to quit.")
    animal_name=input("tell me you animal's name:")
    if animal_name=='q':
        break

    pet1=make_pet(animal_type,animal_name)
    print(pet1)
```

```
enter 'q' at any time to quit.
enter 'q' at any time to quit.
{'pet_type': 'cat', 'pet_name': 'happy'}
enter 'q' at any time to quit.
```

def + list(单)

In [146…

```
def greet_users(names):
    for name in names:
        msg="Hello, " + name.title() + "!"
        print(msg)

usernames=['mica','anna','sofia']
greet_users(usernames)
```

```
Hello, Mica!
Hello, Anna!
Hello, Sofia!
```

def + list(多)+pop() / append()

In [151…

```
# 解法1 薄弱题
unprinted_designs=['iphone case', 'robot pendant', 'dodecahedron']
completed_models=[]

while unprinted_designs:
    current_design=unprinted_designs.pop()

    print("Printed_designs:" + current_design)
    completed_models.append(current_design)
```

```
print("\nThe following models have been printed:")
for completed_model in completed_models:
    print(completed_model)
```

```
Printed_designs:dodecahedron
Printed_designs:robot pendant
Printed_designs:iphone case

The following models have been printed:
dodecahedron
robot pendant
iphone case
```

In [156…

```
# 解法2 薄弱题
def print_models(unprinted_designs,completed_models):
    while unprinted_designs:
        current_design=unprinted_designs.pop()

        print("Printed model: " + current_design)
        completed_models.append(current_design)

def show_completed_models(completed_models):
    print("\nThe following models have been printed:")
    for completed_model in completed_models:
        print(completed_model)

unprinted_designs = ['iphone case', 'robot pendant', 'dodecahedron']
completed_models=[]

print_models(unprinted_designs,completed_models)
show_completed_models(completed_models)
```

```
Printed model: dodecahedron
Printed model: robot pendant
Printed model: iphone case

The following models have been printed:
dodecahedron
robot pendant
iphone case
```

def +list(copy)--不改变原列表

In [157…

```
def print_models(unprinted_designs,completed_models):
    while unprinted_designs:
        current_design=unprinted_designs.pop()

        print("Printed model: " + current_design)
        completed_models.append(current_design)

def show_completed_models(completed_models):
    print("\nThe following models have been printed:")
    for completed_model in completed_models:
        print(completed_model)

unprinted_designs = ['iphone case', 'robot pendant', 'dodecahedron']
completed_models=[]

# 加了符号:
```

```
print_models(unprinted_designs[:],completed_models)
show_completed_models(completed_models)
```

```
Printed model: dodecahedron
Printed model: robot pendant
Printed model: iphone case

The following models have been printed:
dodecahedron
robot pendant
iphone case
```

Excercise

In [158…

```
# 8-9
def show_magicians(names):
    for name in names:
        print(name.title())

magicians=['mica','anna','jessica']
show_magicians(magicians)
```

```
Mica
Anna
Jessica
```

def + list 顺序输出 + 描述语(改变原列表)

In [17]:

```
# 8-10 解法1 薄弱题
def show_magicians(names):
    for name in names:
        print(name.title())

def make_great(names):
    for i in range(len(names)):
        names[i]="the Great " + names[i]

magicians=['mica','anna','jessica']
make_great(magicians)
show_magicians(magicians)
```

```
The Great Mica
The Great Anna
The Great Jessica
```

def + list 逆序输出 + 描述语(改变原列表)

In [18]:

```
# 8-10 解法2 薄弱题
def show_magicians(names):
    for name in names:
        print(name.title())

def make_great(names):
    great_magicians = []
```

```
    while names:  # names非空时执行
        current_magician = names.pop()     # jessica-anna-mica
        great_magician = "the Great " + current_magician  # the Great + (jessica
        great_magicians.append(great_magician) # the Great + (jessica-anna-mica)

    # 把加好问候语的名字放回列表
    for great_magician in great_magicians:
        names.append(great_magician)

magicians = ['mica','anna','jessica']

make_great(magicians) # 先加问候语
show_magicians(magicians) # 再输出
```

```
The Great Jessica
The Great Anna
The Great Mica
```

def + list 顺序输出 + 描述语(改变原列表)

In [19]:
```
# 8-10 顺序不变 薄弱题
def show_magicians(names):
    for name in names:
        print(name.title())

def make_great(names):
    great_magicians = []

    while names:
        current_magician = names.pop()
        great_magician = "the Great " + current_magician
        great_magicians.append(great_magician)

    # 倒序放回去，保持原来顺序
    while great_magicians:
        names.append(great_magicians.pop())

magicians = ['mica','anna','jessica']

make_great(magicians)
show_magicians(magicians)
```

```
The Great Mica
The Great Anna
The Great Jessica
```

def + list copy输出 + 描述语(不改变原列表)

In [20]:
```
# 薄弱题
def show_magicians(names):
    for name in names:
        print(name.title())

def make_great(names):
    # 建立一个空列表，存放加了称号的魔术师
```

```python
    great_magicians = []

    # 把每个名字取出来，加上 "the Great"
    while names:
        current = names.pop()
        great_name = "the Great " + current
        great_magicians.append(great_name)

    # 返回新列表
    return great_magicians


# 原始列表
magicians = ['mica', 'anna', 'jessica']

# 重点：传副本！不修改原列表
great_magicians = make_great(magicians[:])

# 显示原始列表（没变）
print("Original magicians:")
show_magicians(magicians)

# 显示新列表（加了称号）
print("\nGreat magicians:")
show_magicians(great_magicians)
```

```
Original magicians:
Mica
Anna
Jessica

Great magicians:
The Great Jessica
The Great Anna
The Great Mica
```

def + *任意参数(非字典一颗星)

In [197…

```python
def make_pizza(*toppings):
    print(toppings)

make_pizza('pepperoni')
make_pizza('mushrooms', 'green peppers', 'extra cheese')
```

```
('pepperoni',)
('mushrooms', 'green peppers', 'extra cheese')
```

In [201…

```python
def make_pizza(*toppings):
    print("Making pizza with the following toppings:")
    for topping in toppings:
        print("-" + topping)

make_pizza('mushrooms', 'green peppers', 'extra cheese')
```

```
Making pizza with the following toppings:
-mushrooms
-green peppers
-extra cheese
```

def + *任意参数+ 位置参数

In [212…

```
def make_pizza(size,*toppings):
    print("\nMaking a " + str(size) + "-inch pizza witn the following toppings:"
    for topping in toppings:
        print("- " + topping)

make_pizza(16, 'green peppers', 'extra cheese')
make_pizza(12, 'green peppers', 'extra cheese','mushrooms')
```

```
Making a 16-inch pizza witn the following toppings:
- green peppers
- extra cheese

Making a 12-inch pizza witn the following toppings:
- green peppers
- extra cheese
- mushrooms
```

def + **字典参数(两颗星)

In [215…

```
def build_profile(first,last,**user_info):
    profile={}
    profile['first_name']=first
    profile['last_name']=last
    for key,value in user_info.items():
        profile[key]=value
    return profile

user_profile=build_profile('albert', 'einstein',location='princeton',field='phys
print(user_profile)
```

```
{'first_name': 'albert', 'last_name': 'einstein', 'location': 'princeton', 'fiel
d': 'physics'}
```

In [217…

```
def visit(first,last,**visitor_infos):
    visit_cloud={}
    visit_cloud['first_name']=first
    visit_cloud['last_name']=last

    for key,value in visitor_infos.items():
        visit_cloud[key]=value
    return visit_cloud

visitor=visit('mica','liang',country='China',city='Beijing')
print(visitor)
```

```
{'first_name': 'mica', 'last_name': 'liang', 'country': 'China', 'city': 'Beijin
g'}
```

Exercise

In [227…

```
# 8-12
def sandwich(*toppings):
```

```
    print("\n Making a andwich witn the following toppings:")
    for topping in toppings:
        print("- " + topping)

sandwich('egg','mango')
sandwich('egg','mango','watermelon')
```

```
 Making a andwich witn the following toppings:
- egg
- mango

 Making a andwich witn the following toppings:
- egg
- mango
- watermelon
```

In [234…

```
# 8-13
def user_profile(first,last,**user_infos):
    profile={}
    profile['first_name']=first
    profile['last_name']=last

    for key,value in user_infos.items():
        profile[key]=value
    return profile

myself=user_profile('mica','liang',country='china',hobby='reading')
print(myself)
```

```
{'first_name': 'mica', 'last_name': 'liang', 'country': 'china', 'hobby': 'readin
g'}
```

In [240…

```
# 8-14
def make_car(manufacturer,model,**car_infos):
    car_dict={}
    car_dict['manufacturer']=manufacturer
    car_dict['model']=model

    for key,value in car_infos.items():
        car_dict[key]=value
    return car_dict

car1 = make_car('subaru', 'outback', color='blue', tow_package=True)
print(car1)

car2 = make_car('bmw', 'model_s', color='red', engine='oversea')
print(car2)
```

```
{'manufacturer': 'subaru', 'model': 'outback', 'color': 'blue', 'tow_package': Tr
ue}
{'manufacturer': 'bmw', 'model': 'model_s', 'color': 'red', 'engine': 'oversea'}
```

founction → module

In [ ]:

```
# 已练习
# Crash_module
# Crash_module_test
```

In [ ]:
```
# Styling function(规范function书写)
# 1. function name 必须小写，大于一个词，需要下划线，如：  make_Pizza.
# 2. 加备注在function的的下面来解释功能 如：""" To descripe the full name of user
# 3.要有arguments参数
# 4.参数是默认值时，等号两边不能加空格 如：function_name(name, age=12)
```

In [ ]:

In [ ]: